UNIVERSIDAD AUTÓNOMA DE ZACATECAS

UNIDAD ACADÉMICA DE INGENIERIA ELÉCTRICA

INGENIERÍA DE SOFTWARE

ANÁLISIS Y DISEÑO DE ALGORÍTMOS

ANÁLISIS DE ALGORITMOS DE ORDENAMIENTO

CASTILLO RODRÍGUEZ EVELYN DANIELA
42402146

Análisis de algoritmos de almacenamiento. Este documento contiene el análisis de cada algoritmo presente en la página **Visualgo**, se presenta el desglose de la notación **big O**, el graficado del desempeño y el análisis comparativo de todos los métodos de ordenamiento

- **BUBBLE SORT**

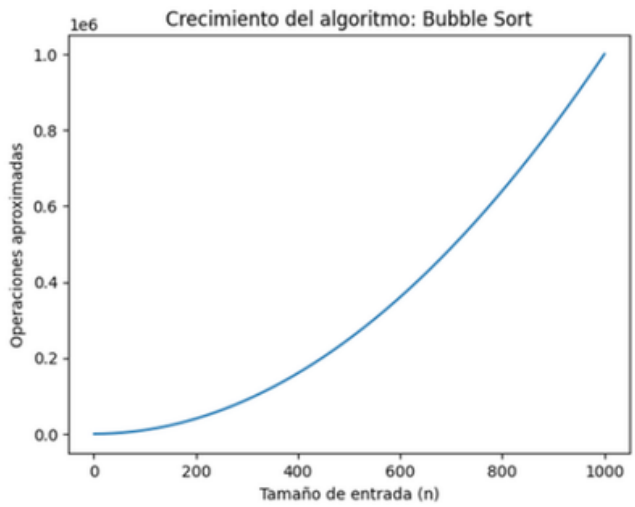
Funciona comparando pares, si el de la izquierda es más grande que el de la derecha se intercambian de lugar. Ese proceso se repite hasta que el valor mas grande llegue al final.

```
void bubbleSort(int arr[], int n) {
    bool swapped;
    // Bucle externo-> O(n)
    for (int i = 0; i < n - 1; i++) {
        swapped = false;

        // Bucle interno-> O(n)
        // En cada pasada 'i', el rango de comparación se reduce
        for (int j = 0; j < n - i - 1; j++) {

            // Comparación: operación de tiempo constante -> O(1)
            if (arr[j] > arr[j + 1]) {

                // Intercambio (Swap): operación de tiempo constante -> O(1)
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }

        // Optimización: si no hubo intercambios, el arreglo ya está ordenado
        if (!swapped) break;
    }
}
```

En la primera pasada hace *n-1* comparaciones, en la segunda *n-2*, y así sucesivamente. La suma total de comparaciones sigue la progresión: *(n-1)+(n-2)+...+1=(n(n-1))/2.* Al expandir (n^2-n)/2 y eliminar los términos de menor orden, obtenemos una complejidad asintótica de O(n^2).

- Peor caso: ***O(n²)*** El arreglo esta en orden inverso, se realizan todas las comparaciones e intercambios posibles
- Mejor caso: ***O(n)*** El arreglo ya esta ordenado, solo se realiza una pasada de n comparaciones
- Caso promedio: ***O(n²)*** Los elementos están en orden aleatorio

Gráfica de crecimiento del algoritmo:



- **SELECTION SORT**

Busca el elemento más pequeño de todos y lo coloca al inicio, después busca el siguiente más pequeño de los restantes y lo pone en segundo lugar, y continúa así sucesivamente,

```
void selectionSort(int arr[], int n) {
    // Bucle externo: Se ejecuta n-1 veces -> O(n)
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;

        // Bucle interno: Busca el mínimo en la parte desordenada.
        // Se ejecuta una cantidad decreciente de veces (n-i-1) -> O(n)
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {
                min_idx = j; // Comparación constante -> O(1)
            }
        }

        // Intercambio: Se realiza una sola vez por cada iteración del bucle externo.
        // Operación de tiempo constante -> O(1)
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
```

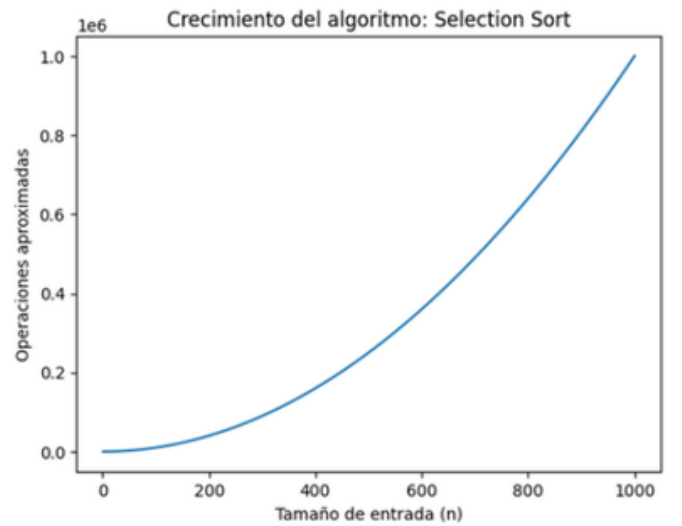
En la primera pasada, comparamos el primer elemento con los *n-1* restantes. En la segunda, buscamos el mínimo entre *n-2* elementos. El número total de comparaciones sigue la misma progresión aritmética que el Insertion Sort en su peor caso:

$$(n-1)+(n-2)+(n-3)+\ldots+1=(n(n-1))/2$$

Al expandir la fórmula obtenemos $n^2-n/2$. Según el modelo teórico, descartamos las constantes y los términos de menor orden.
Resultado Final: ***O(n^2)*** (Complejidad Cuadrática).

- Peor caso: $\boldsymbol{O(n^2)}$ El arreglo está en orden inverso, se realizan todas las comparaciones y un intercambio en cada paso
- Mejor caso: $\boldsymbol{O(n^2)}$ El arreglo ya esta ordenado

Gráfica de crecimiento del algoritmo:



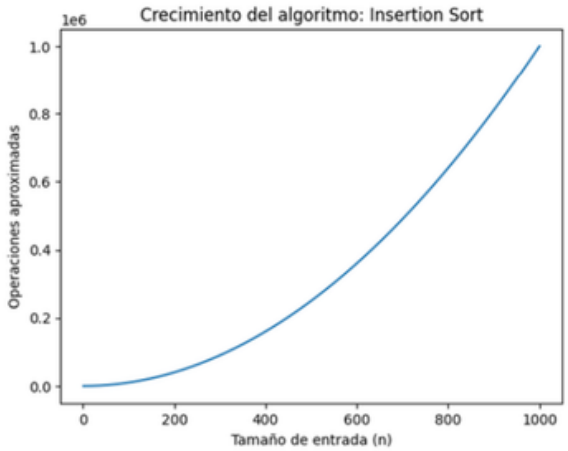
- **INSERTION SORT**

Comienza desde el índice 1, y lo compara con los elementos de la izquierda y desplaza a la derecha los elementos que sean mayores.

```
void insertionSort(int arr[], int n) {
    // 1. Bucle externo: Se ejecuta n-1 veces (O(n))
    for (int i = 1; i < n; i++) {
        int key = arr[i];          // Operación constante (O(1))
        int j = i - 1;             // Operación constante (O(1))

        /* 2. Bucle interno: Su complejidad depende del orden del arreglo.
           En cada iteración 'i', puede ejecutarse hasta 'i' veces. */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];   // Desplazamiento (O(1) por cada movimiento)
            j--;                   // Decremento (O(1))
        }

        arr[j + 1] = key;          // Inserción final (O(1))
    }
}
```

- Peor caso: ***O(n²)*** El arreglo está en orden inverso, para cada elemento *i*, el bucle interno debe realizar *i* comparaciones y desplazamientos.
- Mejor caso: ***O(n)*** El arreglo ya esta ordenado

Gráfica de crecimiento del algoritmo:



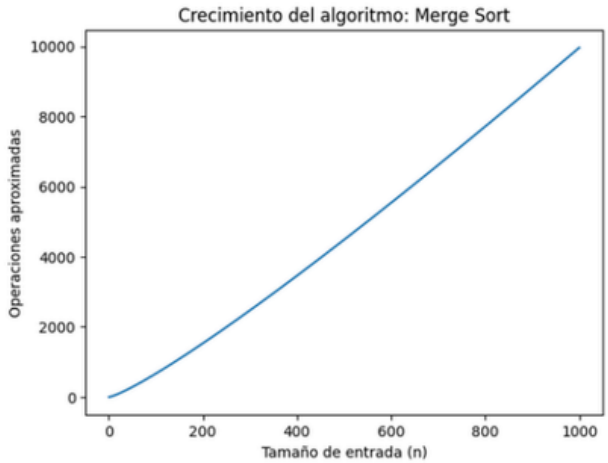
- **MERGE SORT**

"Divide y vencerás", utiliza el método de dividir la lista a la mitad, luego esa mitad a la mitad. Se agrupan en pares, en cuatro, para ordenarse

```
// Función para mezclar dos mitades
void merge(int arr[], int l, int m, int r) {
    // Esta parte tiene una complejidad O(n)
    // porque debe recorrer todos los elementos para compararlos y mezclarlos.
    // Es el "corazón" del trabajo pesado.
}

// Función principal recursiva
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2; // Punto medio

        // Estas llamadas representan la división logarítmica: O(log n)
        mergeSort(arr, l, m);    // Mitad izquierda
        mergeSort(arr, m + 1, r); // Mitad derecha

        // Llamada a la mezcla
        merge(arr, l, m, r);
    }
}
```

- Peor caso: ***O(n log n)*** El arreglo está totalmente desordenado o invertido
- Mejor caso: ***O(n log n)*** El arreglo ya esta ordenado

Gráfica de crecimiento del algoritmo:



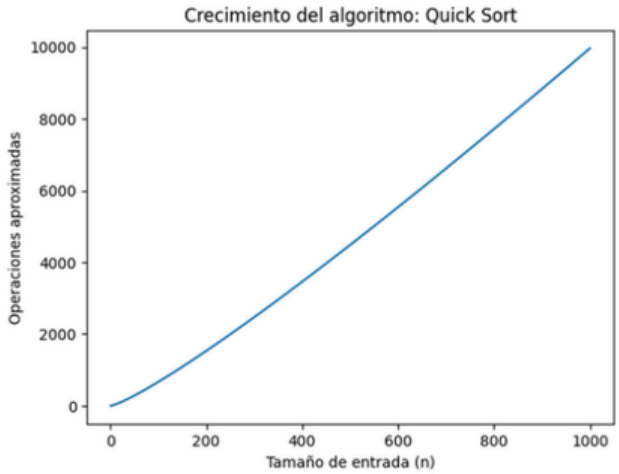
- **QUICK SORT**

"Divide y vencerás", se centra en la selección de un elemento llamado *pivote,* para particionar el arreglo en dos sub-arreglos:

- Elementos menores o iguales al pivote a la izquierda
- Elementos mayores al pivote a la derecha

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Selección del pivote (O(1))
    int i = (low - 1);     // Índice del elemento más pequeño

    // Este bucle recorre n elementos del sub-arreglo actual
    // Complejidad de esta sección: O(n)
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]); // Intercambio (O(1))
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

// Función recursiva QuickSort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // La partición toma O(n) en cada nivel
        int pi = partition(arr, low, high);

        // Llamadas recursivas:
        // En el mejor caso, cada una procesa n/2 elementos
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

- Peor caso: ***O(n²)*** El pivote seleccionado es el elemnto más pequeño o el más grande
- Mejor caso: ***O(n log n)*** El pivote siempre divide el arreglo en dos mitades aproximadamente iguales

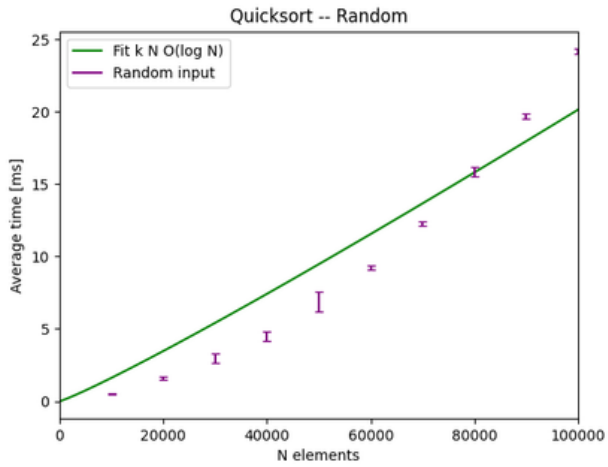


- **RANDOM-QUICK SORT**

Variante del algoritmo *Quick sort,* la diferencia esta en la elección del pivote, en lugar de elegir siempre el primer o el último elemento se selecciona uno al azar

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // El pivote ya fue movido aquí aleatoriamente
    int i = (low - 1);

    for (int j = low; j <= high - 1; j++) { // Se ejecuta n veces: O(n)
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]); // Intercambio constante: O(1)
        }
    }
    swap(&arr[i + 1], &arr[high]); // O(1)
    return (i + 1);
}

// Función principal de Quick Sort Aleatorio
void randomizedQuickSort(int arr[], int low, int high) {
    if (low < high) { // Condición de parada
        // 1. Selección aleatoria del pivote y swap con el último: O(1)
        int random = low + rand() % (high - low);
        swap(&arr[random], &arr[high]);

        // 2. Partición: Divide el problema: O(n)
        int pi = partition(arr, low, high);

        // 3. Llamadas recursivas: Se repiten log n veces (promedio)
        randomizedQuickSort(arr, low, pi - 1);
        randomizedQuickSort(arr, pi + 1, high);
    }
}
```

- Peor caso: ***O(n²)*** El pivote seleccionado siempre se elige como el más pequeño o más grande del sub arreglo, quedando en una división desequilibrada (un subarreglo de tamaño n-1 y otro de 0), generando n niveles de recursión
- Mejor caso: ***O(n log n)*** El pivote seleccionado aleatoriamente divide el arreglo en dos mitades, generando un arbol de llamadas con una profundidad de log n,

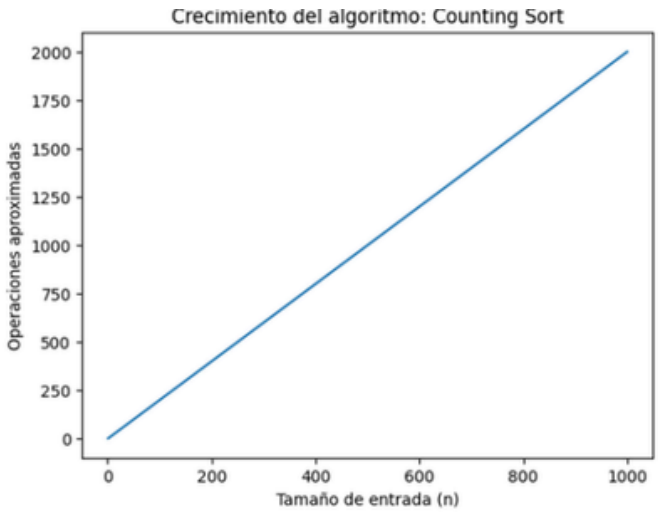


- **COUNTING SORT**

Utiliza las propiedades de los números enteros para contar la frecuencia de cada valor en un rango determinado y determinar su posición final directamente

```python
def counting_sort(arr):
    if not arr: return arr

    # 1. Encontrar el rango (k) -> O(n)
    max_val = max(arr)

    # 2. Inicializar arreglo de conteo -> O(k)
    count = [0] * (max_val + 1)

    # 3. Contar frecuencias -> O(n)
    for num in arr:
        count[num] += 1

    # 4. Calcular posiciones acumuladas -> O(k)
    for i in range(1, len(count)):
        count[i] += count[i-1]

    # 5. Construir arreglo de salida -> O(n)
    output = [0] * len(arr)
    for num in reversed(arr):
        output[count[num] - 1] = num
        count[num] -= 1

    return output
```

- Peor caso: ***O(k)*** El rendimiento decae drásticamente si k es mucho mayor que n ya que el tiempo O(k) dominaría la ejecución
- Mejor caso: ***O(n+k)*** Ocurre siempre, si los datos están ordenados o no, ya que el algoritmo siempre debe recorrer todo el arreglo n y todo el rango k

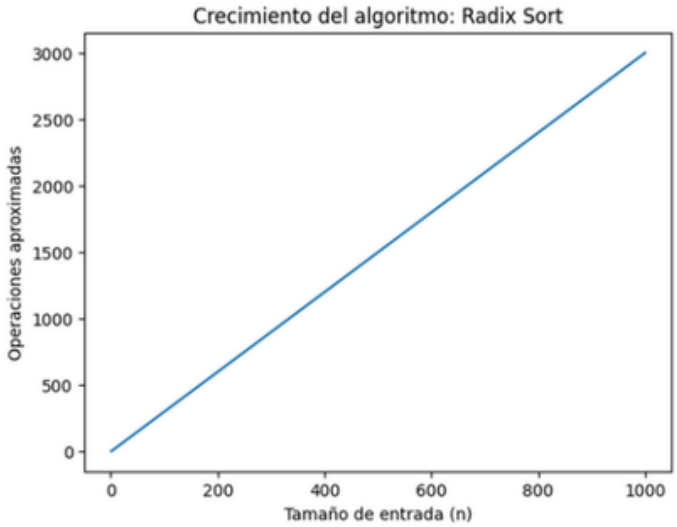


- **RADIX SORT**

Clasifica los números procesando sus dígitos de forma individualmente desde el dígito menos significativo (LSD) hasta el más significativo (MSD). Utiliza un sub-algoritmo de ordenamiento estable (comúnmente Counting Sort) para organizar los números en cada posición decimal.

```
// d = número de dígitos, n = tamaño del arreglo, k = base (10)
void radixSort(int arr[], int n) {
    int m = getMax(arr, n); // O(n) para encontrar el número máximo

    // Bucle externo: se ejecuta d veces (una por cada dígito)
    // Complejidad: O(d)
    for (int exp = 1; m / exp > 0; exp *= 10) {

        // --- INICIO SUB-ALGORITMO (COUNTING SORT) ---
        // Se ejecuta en cada iteración del bucle for
        // Complejidad de esta sección: O(n + k)

        int output[n];
        int count[10] = {0};

        for (int i = 0; i < n; i++) // Contar ocurrencias: O(n)
            count[(arr[i] / exp) % 10]++;

        for (int i = 1; i < 10; i++) // Ajustar conteo: O(k)
            count[i] += count[i - 1];

        for (int i = n - 1; i >= 0; i--) { // Construir arreglo de salida: O(n)
            output[count[(arr[i] / exp) % 10] - 1] = arr[i];
            count[(arr[i] / exp) % 10]--;
        }

        for (int i = 0; i < n; i++) // Copiar al arreglo original: O(n)
            arr[i] = output[i];
    }
}
```

- Peor caso: ***O(d(n+k))*** Ocurre sin importar el orden ya sea si todos los números tienen muchos dígitos (d es grande), el rendimiento se ve afectado
- Mejor caso: ***O(d(n+k))*** Ocurre siempre, si los datos están ordenados o no, ya que el algoritmo siempre debe recorrer todo el arreglo n y todo el rango k



- **CONCLUSIÓN**

Algoritmo	Mejor Caso	Peor Caso	Caracteristicas Experimentales
Bubble Sort	O(n)	O(n^2)	Realiza múltiples intercambios adyacentes. Es el más lento en la práctica.
Insertion Sort	O(n)	O(n^2)	Eficiente para arreglos casi ordenados. El documento confirma experimentalmente que un aumento de 20x en n puede generar un incremento de 329x en movimientos.
Selection Sort	O(n^2)	O(n^2)	Minimiza el número de intercambios, pero siempre realiza el mismo número de comparaciones.

Merge Sort	O(n \log n)	O(n \log n)	Estable y predecible. Divide el problema recursivamente, lo que garantiza un rendimiento constante independientemente del orden inicial.
Quick Sort	O(n \log n)	O(n^2)	Suele ser el más rápido en la práctica debido a factores de hardware (caché), aunque puede degradarse si el pivote es malo.
Random-Quick	O(n \log n)	O(n \log n)	Variante que elige el pivote al azar para evitar el peor caso casi por completo, manteniendo una alta eficiencia.

Counting Sort	O(n + k)	Extremadamente rápido pero limitado a números enteros dentro de un rango k conocido. Requiere memoria adicional considerable.
Radix Sort	O(d(n + k))	Ideal para ordenar números grandes o cadenas de texto. Su rendimiento depende de la cantidad de dígitos (d) y la base (k).